Assignment 4, explained

California Housing — from split to Random Forest

A plain-language walkthrough of every question,
with the numbers from the solution notebook.

SVNIT · Department of Artificial Intelligence
Machine Learning · u24ai028

The whole assignment in one page

You are given about 20,640 districts from the 1990 California census. Each row is one district, not one house. The job is to predict median_house_value from location, age, rooms, population, income, and how close the district is to the ocean.

The work has three stages. First, split the data so the test set looks like the real world. Second, clean the columns (fill missing numbers, scale them, turn ocean_proximity into 0/1 columns) and try three models. Third, tune the best model, score it once on the test set, and also solve a tiny linear regression by hand with the Normal Equation.

Columns

- longitude, latitude — where the district is
- housing_median_age — typical age of houses there
- total_rooms, total_bedrooms, population, households — size of the district
- median_income — typical income, in units of $10,000
- ocean_proximity — text: <1H OCEAN, INLAND, ISLAND, NEAR BAY, NEAR OCEAN
- median_house_value — the number you predict

One warning you will see in the plots: house values are capped at $500,001. Anything more expensive was recorded as that number. The model cannot learn prices above the cap.

Q1. Split by income, not at random

A random 80/20 split can, by luck, put too many rich districts in the test set. Income is the strongest clue about price, so the test set must have the same mix of incomes as the full data.

median_income is a continuous number, and stratified splitting needs groups. So you cut it into 5 bins called income_cat:

bins: 0 – 1.5 – 3.0 – 4.5 – 6.0 – infinity labels: 1, 2, 3, 4, 5

StratifiedShuffleSplit(test_size=0.2, random_state=42) then keeps each bin's share almost identical in train and test.

income_cat	Train count	Test count	Share
1 (poorest)	657	165	about 4%
2	5,265	1,316	about 32%
3	5,789	1,447	about 35%
4	2,911	728	about 18%
5 (richest)	1,890	472	about 11%

Train has 16,512 districts. Test has 4,128. After the split, income_cat is dropped. It was only a tool for splitting. The model still sees the original median_income.

Say this: I stratified on income bins so the test set has the same rich/poor mix as the full data. A plain random split might not.

Q2. One pipeline, two kinds of columns

Models need numbers, and they need every row filled in. total_bedrooms is missing for 158 training districts. ocean_proximity is text. A ColumnTransformer sends each kind of column down its own

path.

Numeric path (8 columns)

- SimpleImputer(strategy="median") fills blanks with the median. The median is not pulled around by a few huge districts.
- StandardScaler subtracts the mean and divides by the standard deviation, so every column is on a similar scale.

Category path (ocean_proximity)

- OneHotEncoder makes 5 columns, one per category. A district near the bay is [0, 0, 0, 1, 0], not the number 3.
- handle_unknown="ignore" means a brand-new category at test time becomes all zeros instead of crashing. ISLAND is very rare, so this matters.

Resulting training array: shape (16512, 13). That is 16,512 rows and 13 columns = 8 scaled numbers + 5 one-hot columns.

fit_transform is used on the training set only. The test set gets transform. The median and the mean/std must be learned from training rows. If you compute them on the test set too, the test set has leaked into the preprocessing.

Say this: 13 columns: 8 scaled numeric features plus 5 yes/no ocean categories. I fit the imputer and scaler on train only.

Q3. Three models, scored with 5-fold CV

Cross-validation cuts the training set into 5 folds. Each model is trained on 4 folds and scored on the leftover fold, five times. The reported number is the average RMSE, plus how much that score wobbles (the std).

RMSE is the typical prediction error in dollars. Lower is better. sklearn returns the negative of RMSE because it always tries to maximise a score. The notebook flips the sign back.

Model	CV RMSE
Linear regression	$69,218 ± $690
Decision tree	$70,629 ± $1,448
Random forest (100 trees)	$49,942 ± $761

- Linear regression draws one flat relationship. Price depends on location and income in curves and jumps, so a straight fit leaves about $69k of error.
- One decision tree can memorise the training districts. On a new fold it is worse than the straight line, and its score jumps around (largest std).
- A random forest averages many trees. Each tree sees a random sample of rows and a random subset of columns. Averaging cancels the wild mistakes. It wins, at about $50k typical error.

Say this: The forest wins because averaging trees cuts the overfitting that kills a single tree, and the splits capture patterns a straight line misses.

Q4. Grid search on the forest

Two knobs are tried. n_estimators is how many trees (50, 100, or 200). max_features is how many columns a split is allowed to look at (2, 4, 6, or 8). That is 3 × 4 = 12 combinations. Each one is

scored with 5-fold CV, so the search trains 60 forests.

Best combination: n_estimators = 200, max_features = 8. Best CV RMSE = $49,015.

More trees make the average steadier. The gain from 100 to 200 trees is only about $140, so extra trees help a little and then flatten out. max_features = 2 makes each tree too weak (RMSE around $51–52k). Allowing 8 features at each split was best inside this grid.

200 and 8 are both at the edge of the grid. The true best setting might be a bit higher. A follow-up search would try max_features of 8, 10, and 12.

Say this: GridSearchCV tried every pair of tree-count and feature-count, kept the one with the lowest cross-validated RMSE, then refit that forest on all of the training data.

Q5. One honest score on the test set

The tuned forest is used once on the 4,128 districts it has never seen. Mean test house value is about $206,258.

Metric	Value	In words
Test RMSE	$46,564	typical error, 22.6% of the mean price
Test MAE	$30,806	average absolute error, 14.9% of the mean
Test R^2	0.8336	83% of the price variation is explained

RMSE is larger than MAE because squaring punishes the big misses. Those big misses sit near the $500,001 cap, where the true price was chopped off. Test RMSE ($46.6k) is close to the CV RMSE ($49.0k), so the tuning did not cheat by fitting the test set.

Say this: On new districts the forest is typically about $47k off, and it explains 83% of the variation in price.

Q6. The Normal Equation, with only NumPy

Linear regression has a closed-form answer. You do not have to search. With a column of 1s added for the intercept, the best weights are:

$$\theta = (X^T X)^{-1} X^T y$$

Here X has only two columns from the training set: median_income and housing_median_age, plus the column of 1s. The same two columns are given to sklearn's LinearRegression. The two answers match to many decimal places. The tiny percent difference (about 10^{-11} %) is rounding error. Both methods minimise the same squared error.

Parameter	Value	Meaning
$\theta 0$ intercept	−10,706.89	baseline value
$\theta 1$ median_income	43,041.08	per +1 income unit
$\theta 2$ housing_median_age	1,776.05	per extra year of age

$$\text{value} = -10{,}706.89 + 43{,}041.08 \times \text{income} + 1{,}776.05 \times \text{age}$$

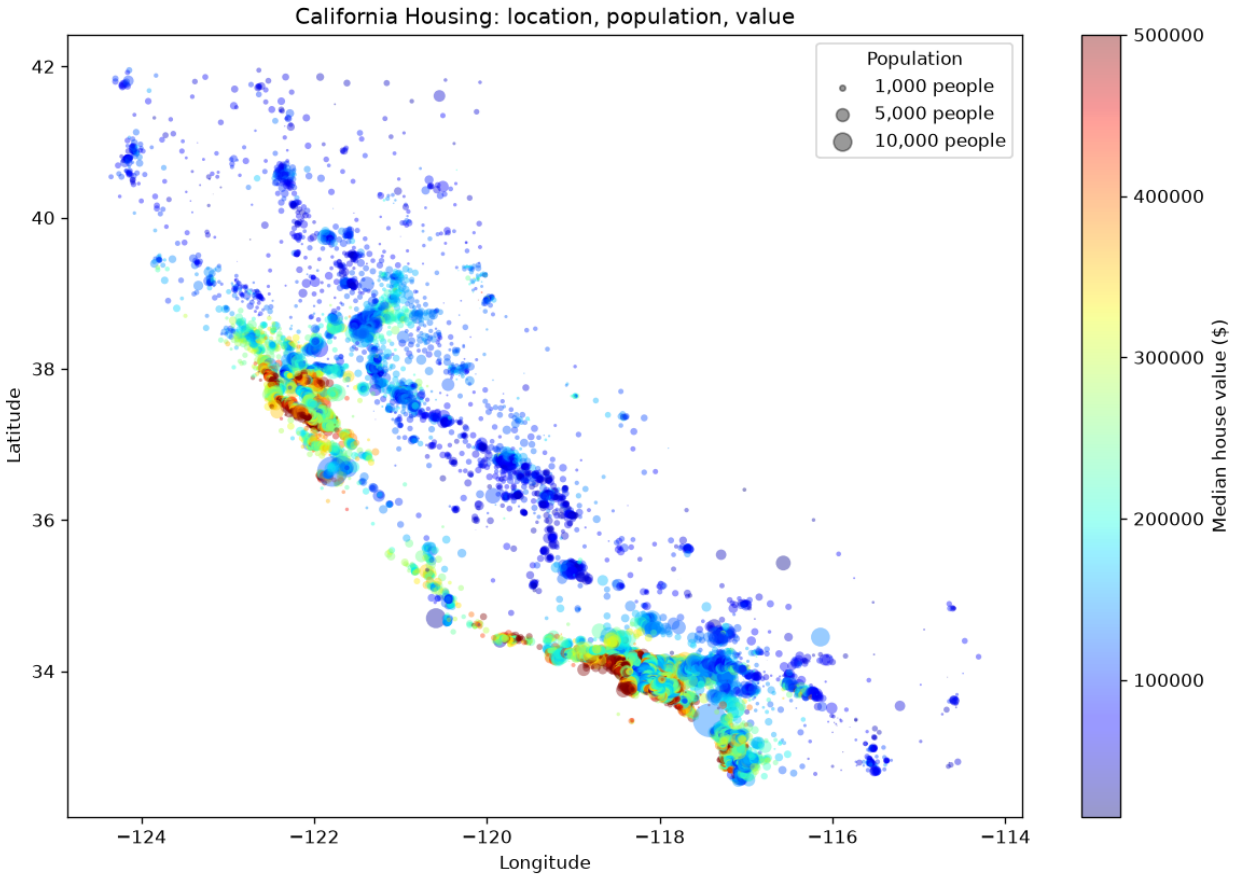
median_income is in units of $10,000. So a $10,000 rise in typical household income goes with about $43,000 more in median house value, if age stays the same.

The Normal Equation is the right tool when there are few columns. Inverting X^TX gets slow as the number of columns grows, and it fails if two columns are copies of each other. Then you use gradient descent instead. That is Assignment 5.

Say this: NumPy and sklearn solve the same least-squares problem, so θ matches. The income slope is about $43k of house value per income unit.

Q7. Where the expensive districts are

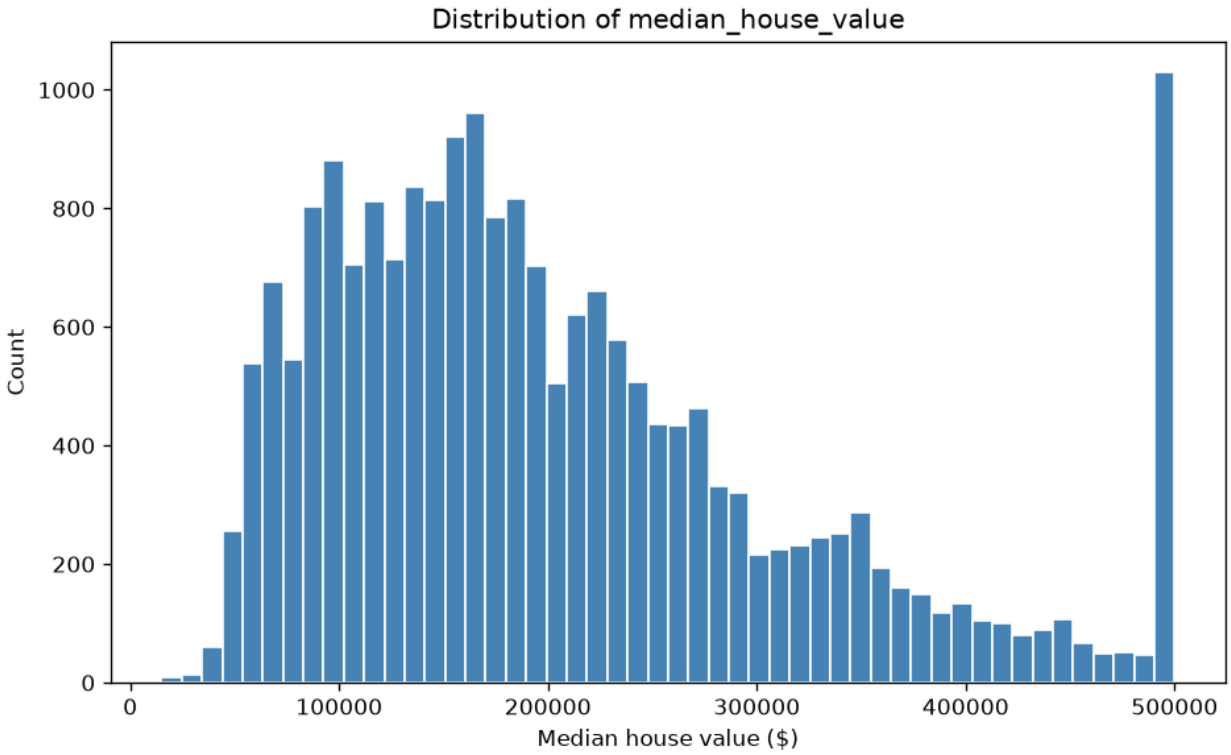
Each point is one district. Horizontal position is longitude, vertical is latitude, so the cloud is the shape of California. Bigger dots mean more people (dot size = population / 100). Colour is median house value. alpha=0.4 lets overlapping dots show through.



Coast and bay districts (Bay Area, Los Angeles, San Diego) are the expensive ones. Inland districts are cheaper. Location should matter in the model.

Q8. The shape of house values

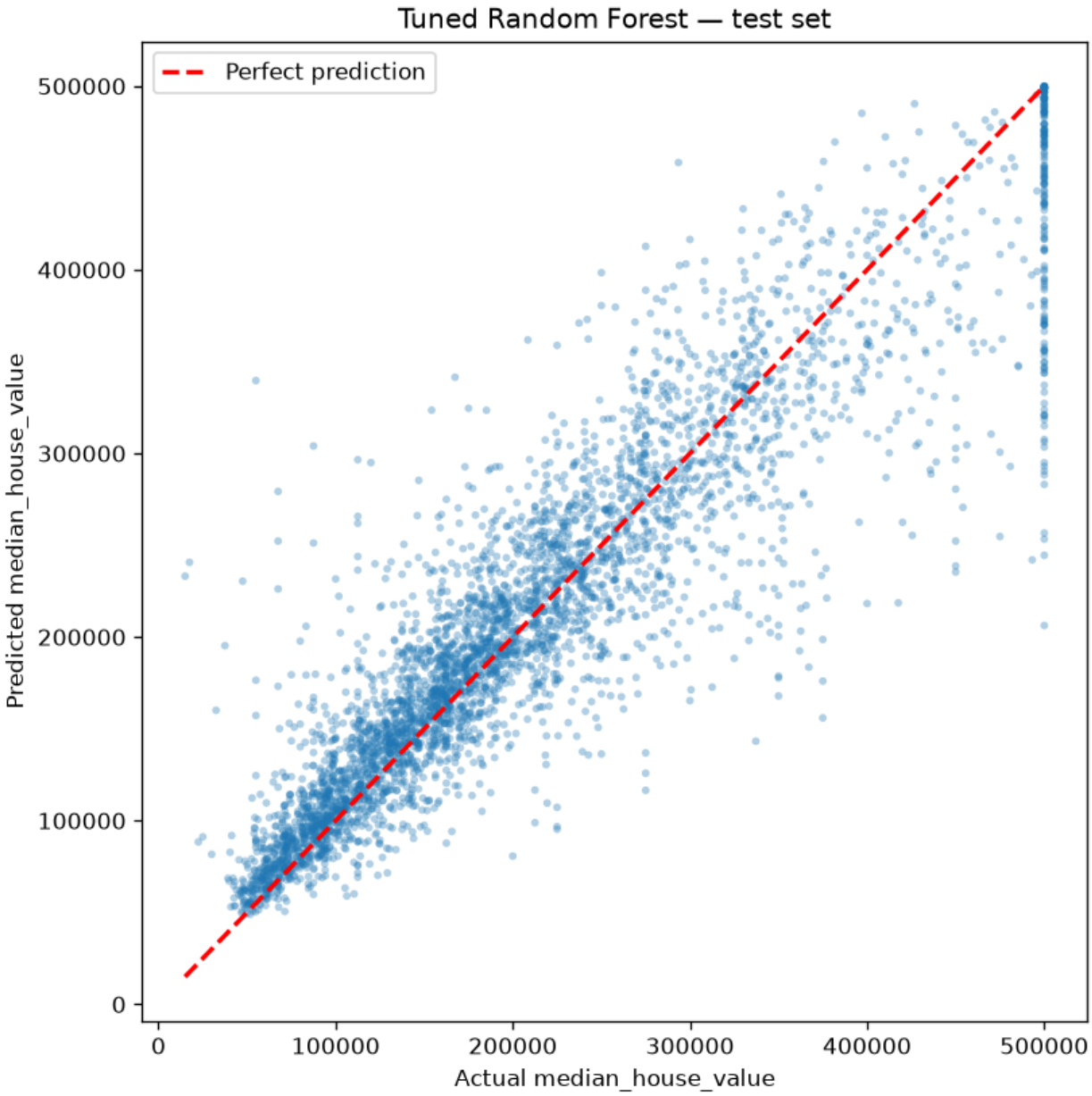
Mean value is $206,856. Median is $179,700. The mean is higher because a long right tail of expensive districts pulls it up. Skewness is 0.98, so the distribution is right-skewed. The tall bar at the far right is the $500,001 cap, not a real pile of houses that all cost exactly that.



Most districts sit between about $100k and $250k. The spike at $500,001 is the census cap.

Q9. Predicted vs actual

Each point is one test district. The horizontal axis is the real median value. The vertical axis is the forest's prediction. The red dashed line is perfect prediction: predicted = actual. Points near that line are good. A vertical strip at $500,001 is the cap: those districts were worth more, but the data says 500001, so the forest predicts below the line.



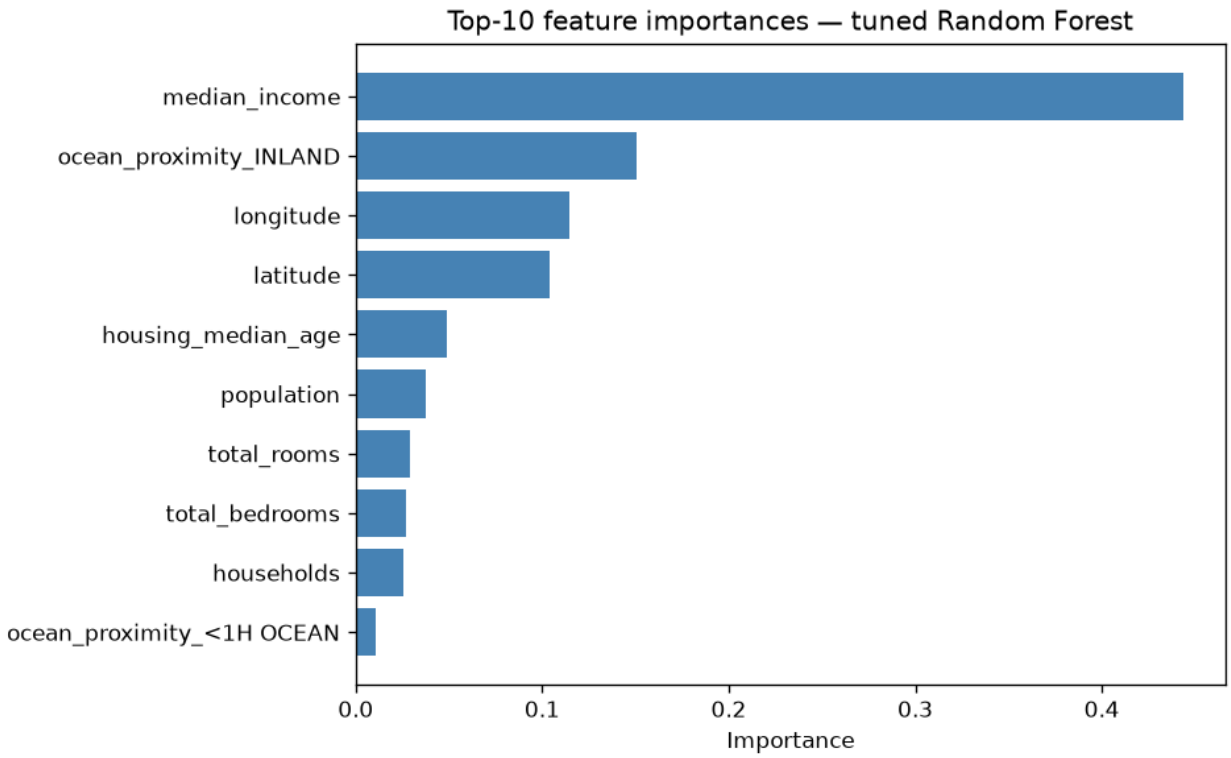
Tight to the diagonal in the middle. The big misses are the capped expensive districts.

Q10. Which columns the forest uses

Feature importance is how much a column reduces error across all the splits in all the trees. The numbers add up to 1. They say "this column was useful," not "this column causes higher prices."

Rank	Feature	Share
1	median_income	44.4%
2	ocean_proximity = INLAND	15.1%
3	longitude	11.4%
4	latitude	10.4%
5	housing_median_age	4.9%

Income plus location (inland flag, longitude, latitude) is almost the whole model. The top 10 features hold 99% of the importance. That matches the map: coast vs inland, and richer districts.



median_income alone is 44% of the forest's importance.

Say this: The forest mostly uses income and location. Inland districts get their own strong signal, which matches the map.

Short answers if you are asked

- Why stratify? So the test set has the same income mix as the full data.
- Why median impute? It is not dragged by extreme districts.
- Why scale? Rooms are in the thousands, income is around 1–15. Linear models care about that.
- Why one-hot, not 0,1,2,3? Those numbers would pretend INLAND < NEAR BAY.
- Why is the tree worse than linear regression? It memorises and then fails on new folds.
- Why does the forest win? Many trees, averaged, keep the useful splits and cancel the noise.
- What is RMSE vs MAE? Both are in dollars. RMSE punishes large errors more.
- What is $R^2 = 0.83$? 83% of the variation in price is explained.
- Why do NumPy and sklearn match in Q6? They minimise the same squared error.
- What is wrong with this data? It is from 1990, values are capped at $500,001, and each row is a district average.